

SOFTWARE ARCHITECTURE AND DESIGN SPECIFICATION

Bank Management System (BMS)

Project	Bank Management System (BMS)
Instructor	Prof. Pavithra
Authors	Aditi Agarwal (PES2UG24CS029) Abhiroop (PES2UG24CS024) Aarush Khullar (PES2UG24CS014) Amilineni Sai Dhyana Anubhav (PES2UG24CS055)
Date	14-09-2026
Status	Draft for Review

Prepared from the project Software Requirements Specification (SRS).

1. Introduction

1.1 Purpose

This document specifies the proposed architecture and design of the Bank Management System (BMS), based on the project SRS. It describes the major modules, component responsibilities, security controls, interfaces, and key transaction flows.

1.2 Scope

The current BMS supports customer registration and login, KYC-based account opening, cash deposits and withdrawals, intra-bank funds transfer, loan applications and approval, mini-statements and receipt/passbook generation, teller-assisted operations, administration, reports, and audit logging. External NEFT, RTGS, SWIFT and physical ATM/card hardware are outside the current release.

1.3 Audience

Project developers, testers, database administrators, bank operations staff, project guide, and evaluators.

1.4 Definitions

BMS – Bank Management System; RBAC – Role-Based Access Control; KYC – Know Your Customer; OTP – One-Time Password; API – Application Programming Interface; TLS – Transport Layer Security; RTM – Requirements Traceability Matrix.

2. Document Overview

2.1 How to use this document

Use this SAD together with the BMS SRS to understand the system structure, module boundaries, security design, API contracts, transaction sequences, and traceability from requirements to design.

2.2 Related Documents

Bank Management System – Software Requirements Specification (SRS), Version 1.0, dated 09-09-2026, including its Requirements Traceability Matrix (RTM).

Revision History

Version	Date	Author	Change Summary	Approval
1.0	09-09-2026	Aditi, Aarush, Sai Dhyam, Abhiroop	Initial SAD prepared from the BMS SRS and course template.	Pending

Approvals

Role	Name	Signature / Date
Course Coordinator	Prof. Pavithra	Pending

3. Architecture

3.1 Goals & Constraints

Goals: secure banking operations, reliable transaction processing, modularity, responsive web access, and measurable performance. Constraints: applicable banking/data-protection requirements, TLS 1.2+, secure password/PIN storage, semester development timeline, and technologies approved by the project guide.

3.2 Stakeholders & Concerns

Stakeholder	Primary Concern
Customer	Simple, fast and secure everyday banking.
Bank Teller	Fast data entry, validation and counter operations.
Branch Manager	KYC/loan approvals, exceptions and reports.
System Administrator	User/role/branch configuration and audit visibility.
Developers & Testers	Modularity, traceability, testability and maintainability.
Compliance	Protection of customer and financial information.

3.3 Component Architecture

Client / Role Layer	Application / Service Layer	Data & Integration Layer
Customer Portal	Authentication & RBAC	Relational Database
Teller / Back Office	Accounts Module	Email/SMS Gateway
Admin Console	Transactions / Transfers	Audit Logs
Manager Dashboard	Loan + Reporting Modules	

Architecture flow: role-specific web interfaces → authentication/RBAC → business modules → database and supporting services.

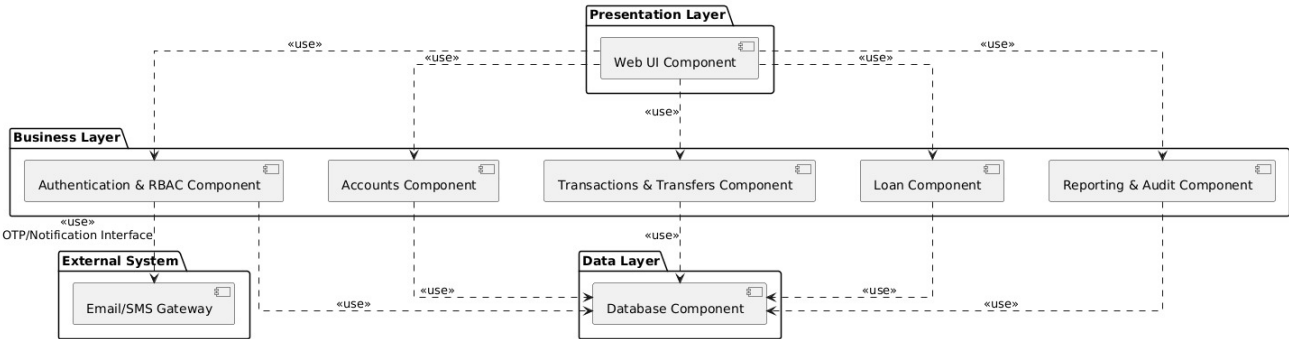


Figure 1: Component UML diagram — Bank Management System architecture.

3.4 Component Descriptions

- Customer Portal – registration, login, account services, deposits/withdrawals, transfers, loans and statements.
- Teller / Back Office – staff-assisted counter operations, account opening and KYC workflows.
- Admin Console – user, role, branch, configuration, reporting and audit visibility.
- Authentication & RBAC – credentials, OTP, session management and role permissions.
- Business Modules – Accounts, Transactions, Transfers and Loans.
- Reporting & Audit – statements, management reports and sensitive-operation logs.
- Relational Database – customer, account, transaction and loan records.

3.5 Chosen Architecture Pattern and Rationale

A modular layered architecture is proposed, consistent with the SRS requirement for separate Authentication, Accounts, Transactions, Loans and Reporting modules. The separation of responsibilities supports maintainability while allowing the modules to use a shared relational database.

3.6 Technology Stack & Data Stores

The selected implementation stack is Node.js + Express.js + React + PostgreSQL. It provides a fast path to a working semester demo, uses a common JavaScript ecosystem across the application and API layers, and supports the security and transaction-consistency requirements identified in the SRS. HTTPS with TLS 1.2 or higher is mandatory.

Layer	Technology	Purpose / Notes
Frontend	React	Role-specific dashboards for Customer, Teller, Manager and Admin; responsive UI.
Backend / API	Node.js + Express.js	REST APIs for authentication, accounts, transactions, transfers, loans and reporting.
Database	PostgreSQL	Customer, account, transaction and loan records; ACID compliance for balance updates.
ORM	Prisma / Sequelize	Schema management and query layer over PostgreSQL.
Authentication	JWT + bcrypt	Session tokens and salted password hashing.
2FA / OTP	Twilio / Nodemailer	OTP delivery for login and high-value transfers.
Transport Security	HTTPS (TLS 1.2+)	Mandatory secure client-server communication.
Hosting / Development	Node runtime + PostgreSQL server	Local or cloud deployment for the project demo.

Technology selection rationale: One language across the full stack reduces context switching for the four-person team; Express.js minimizes boilerplate for authentication, RBAC and transaction flows; the npm ecosystem provides mature packages for bcrypt, JWT, OTP delivery and Prisma/Sequelize; and PostgreSQL provides strong ACID guarantees for correct balance updates in concurrent transactions.

3.7 Risks & Mitigations

Risk	Mitigation
Network/server unavailability	Stable branch connectivity, monitoring and operational checks.
Unauthorized access	RBAC, OTP, secure password hashing and session timeout.
Data exposure	Encryption in transit/at rest and restricted access.
Audit tampering	Protected append-only audit logs.

3.8 Traceability to Requirements

BMS-F-001–004 → Authentication/RBAC; BMS-F-005–007 → Account/KYC; BMS-F-008–010 → Transactions; BMS-F-011–013 → Transfers; BMS-F-014–016 → Loans; BMS-F-017–019 → Reporting/Admin; BMS-NF-001 and BMS-SR-001–005 → performance/security controls.

3.9 Security Architecture

TLS 1.2+ for network connections; salted one-way password hashing; OTP-based 2FA for login and high-value transfers; RBAC for Customers, Tellers, Managers and Admins; audit logs with user ID, timestamp and outcome; configurable idle-session timeout with a default of 10 minutes.

4. Design

4.1 Design Overview

The BMS is a responsive web application with separate Customer, Teller/Back Office, Manager and Administrator views. Core modules share a relational database and use defined application interfaces. The design emphasizes validation, RBAC, transaction consistency, auditability and security.

4.2 UML Sequence Diagrams

Two project-specific flows are derived from the SRS: cash withdrawal and high-value intra-bank transfer.

Flow A — Cash Withdrawal

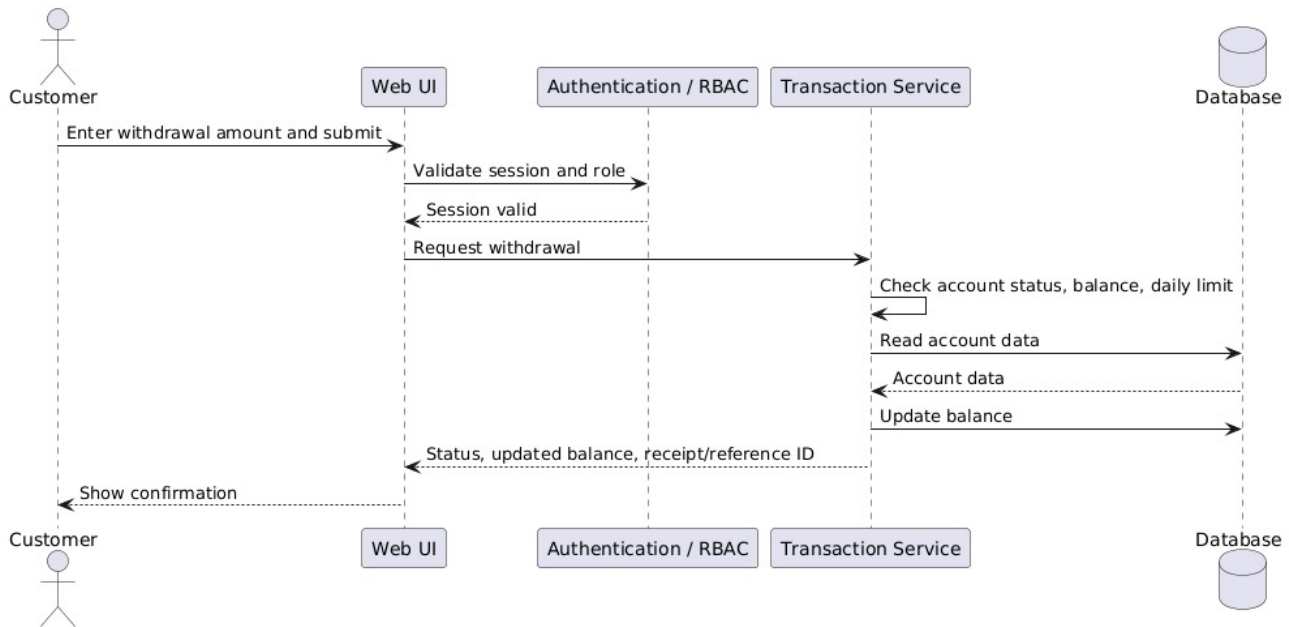


Figure 2: UML sequence diagram — Cash Withdrawal.

Step	Actor / Component	Interaction
1	Customer → Web UI	Enter withdrawal amount and submit.
2	Web UI → Authentication/RBAC	Validate session and role.
3	Transaction Service	Check account status, available balance and daily withdrawal limit.
4	Transaction Service → DB	Read account data and, if valid, update balance.
5	Transaction Service → Web UI	Return status, updated balance and receipt/reference ID.

Flow B — High-Value Intra-Bank Transfer

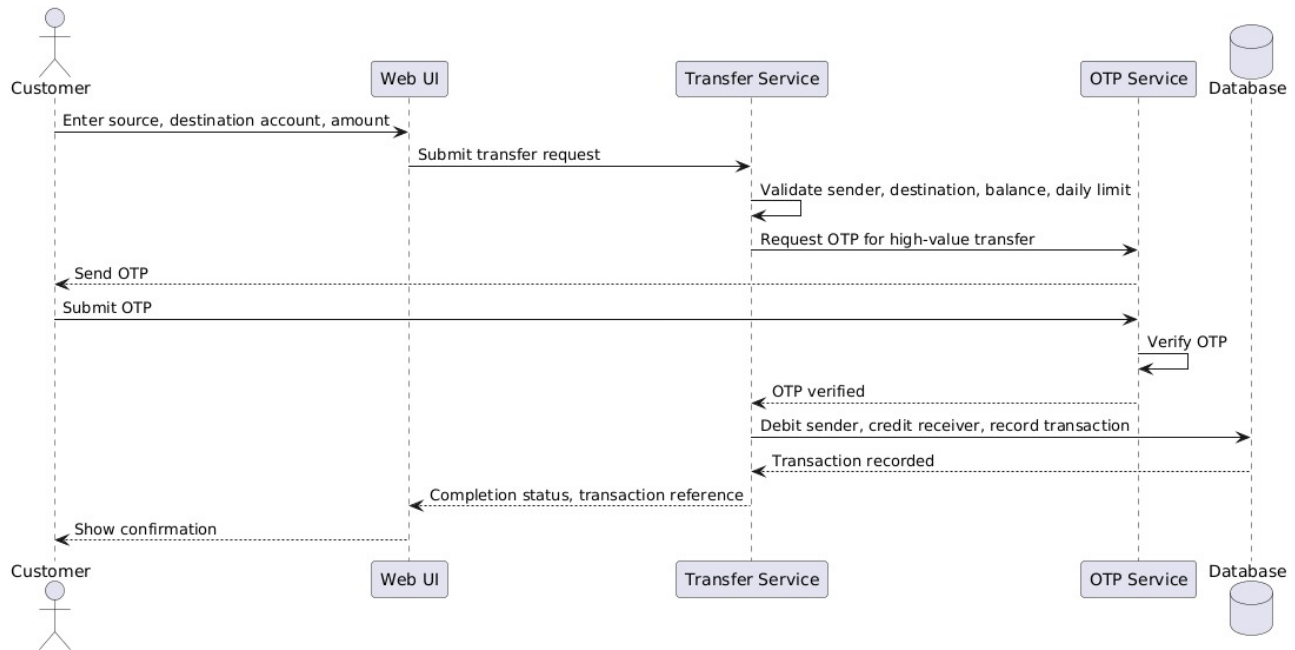


Figure 3: UML sequence diagram — High-Value Intra-Bank Transfer.

Step	Actor / Component	Interaction
1	Customer → Web UI	Enter source account, destination account and amount.
2	Transfer Service	Validate sender, destination, balance and configured daily limit.
3	Transfer Service → OTP Service	Request OTP for a high-value transfer.
4	Customer → OTP Service	Submit and verify the OTP.
5	Transfer Service → DB	Debit sender, credit receiver and record the transaction.
6	Transfer Service → Web UI	Return completion status and transaction reference.

4. Design — Continued

4.3 API Design

The following are proposed design interfaces derived from the SRS requirements; the SRS itself does not prescribe exact endpoint names.

Endpoint	Method	Request	Response / Errors
/auth/login	POST	{customerId, password, otp}	{status, sessionToken}; invalid credentials/OTP or account locked.
/transactions/withdraw	POST	{accountId, amount}	{status, balance, referenceId}; invalid account, insufficient balance or daily limit exceeded.
/transfers	POST	{sourceAccountId, destinationAccountId, amount, otp}	{status, referenceId}; insufficient balance, limit exceeded or invalid OTP.

4.4 Error Handling, Logging & Monitoring

Use clear, non-sensitive error messages. Log authentication attempts, financial transactions and administrative changes with user ID, timestamp and outcome, while protecting logs from tampering. Monitor response time, transaction failures, uptime and security events. Sensitive credentials must never appear in logs.

4.5 UX Design

Responsive desktop/mobile interface; role-specific dashboards; screen-reader-friendly labels and errors; high-contrast accessibility support; and validated teller data-entry screens.

4.6 Open Issues & Next Steps

Implement and verify the selected Node.js + Express.js + React + PostgreSQL stack against the RTM; complete security, performance and accessibility testing; finalize deployment configuration; and consider future payment-gateway or interbank integration. External NEFT/RTGS/SWIFT connectivity remains outside the current release.

5. Appendices

5.1 Glossary

BMS – Bank Management System; UI – User Interface; API – Application Programming Interface; OTP – One-Time Password; KYC – Know Your Customer; DB – Database; RBAC – Role-Based Access Control; TLS/SSL – Transport Layer Security / Secure Sockets Layer; PAN – Permanent Account Number; RTM – Requirements Traceability Matrix.

5.2 References

Bank Management System – Software Requirements Specification (SRS), Version 1.0, dated 09-09-2026.

5.3 Tools

The SRS does not specify a final implementation/design toolset. Tool selection should follow the course/project guide.